



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2026-27

Class:	TE-AI&DS	Semester:	V
Course Code:	2015111	Course Name:	Statistics for Machine Learning and Data Science Lab

Name of Student:	Parth Manoj Gajare
Roll No. :	74
Experiment No.:	6
Title of the Experiment:	Develop and Evaluate Regression Models using Statistical Performance Measures
Date of Performance:	01/09/2026
Date of Submission:	08/09/2026

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty :

Signature :

Date :



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Aim: To develop regression models for predicting a continuous variable using the Pima Indians Diabetes Dataset and evaluate their performance using appropriate statistical metrics.

Objectives

After completing this experiment, the student will be able to:

1. Develop regression models to predict a continuous health-related variable from medical attributes.
2. Evaluate and interpret the performance of regression models using appropriate statistical measures.

Tools Required

- Python 3.x
- Google Colab / Jupyter Notebook
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn

Dataset Details

Dataset: Pima Indians Diabetes Dataset

The dataset contains medical information for **768 female patients**.

For this experiment, **BMI may be selected as the continuous target variable**, while other relevant medical attributes are used as input features.



Procedure

1. Load the Pima Indians Diabetes Dataset.
2. Select a continuous target variable and relevant predictor variables.
3. Check and preprocess missing or invalid values.
4. Divide the dataset into training and testing sets.
5. Develop a suitable regression model.
6. Train the model using the training data.
7. Generate predictions for the test data.
8. Evaluate the model using appropriate performance metrics.
9. Analyze residuals and compare actual and predicted values.
10. Interpret the model results and identify possible limitations.

Description of the Experiment

Regression is a supervised learning technique used to predict a continuous numerical output.

The experiment follows: Dataset, Select Features and Target, Preprocess Data, Split Data, Train Regression Model, Generate Predictions, Evaluate Performance and Interpret Results

Students develop a regression model to estimate a continuous health-related variable based on other available attributes.

Detailed Description of Statistical Methods

Linear Regression

A simple linear regression model can be represented as:

$$y = \beta_0 + \beta_1 x + \epsilon$$

For multiple predictors:



$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \epsilon$$

where:

- y = target variable
- x_i = predictor variables
- β_i = model coefficients
- ϵ = error term

Mean Absolute Error

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

It represents the average absolute prediction error.

Mean Squared Error

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Larger errors are penalized more heavily.

Root Mean Squared Error

$$RMSE = \sqrt{MSE}$$

RMSE is expressed in the same units as the target variable.

Coefficient of Determination

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

indicates the proportion of variation in the target variable explained by the model.



Residual Analysis

A residual is:

$$e_i = y_i - \hat{y}_i$$

Residual analysis helps identify:

- Systematic prediction errors
- Nonlinear patterns
- Unequal variance
- Potential outliers

Code:

```
# 1. Import required libraries
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
from sklearn.preprocessing import StandardScaler
```

```
# 2. Load the dataset
```

```
df = pd.read_csv("diabetes.csv")
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
print("Dataset Shape:")

print(df.shape)

print("\nFirst 5 Rows:")

print(df.head())

print("\nColumn Names:")

print(df.columns.tolist())

# 3. Check missing values

print("\nMissing Values:")

print(df.isnull().sum())

# 4. Replace invalid zero values with NaN

# Zero is not realistic for these medical attributes

invalid_columns = [

    'glucose_concentration',

    'diastolic_blood_pressure',

    'triceps_sf_thickness',

    'serum_insulin',

    'bmi'

]

for col in invalid_columns:

    if col in df.columns:
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
df[col] = df[col].replace(0, np.nan)

print("\nMissing Values After Replacing Invalid Zeros:")

print(df.isnull().sum())

# 5. Fill missing values using median

for col in invalid_columns:

    if col in df.columns:

        df[col] = df[col].fillna(df[col].median())

# 6. Select features and target

# BMI is the continuous target variable

features = [

    'times_pregnant',

    'glucose_concentration',

    'diastolic_blood_pressure',

    'triceps_sf_thickness',

    'serum_insulin',

    'd_pedigree_function',

    'years_of_age'

]

X = df[features]

y = df['bmi']
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
print("\nFeatures:")

print(X.head())

print("\nTarget (BMI):")

print(y.head())

# 7. Split dataset into training and testing data

X_train, X_test, y_train, y_test = train_test_split(

    X, y,

    test_size=0.20,

    random_state=42

)

print("\nTraining Data Size:", X_train.shape)

print("Testing Data Size:", X_test.shape)

# 8. Train Linear Regression model

model = LinearRegression()

model.fit(X_train, y_train)

# 9. Generate predictions

y_pred = model.predict(X_test)

print("\nActual vs Predicted BMI:")

comparison = pd.DataFrame({

    'Actual BMI': y_test.values,

    'Predicted BMI': y_pred
```



```
}}
```

```
print(comparison.head(10))
```

```
# 10. Calculate evaluation metrics
```

```
mae = mean_absolute_error(y_test, y_pred)
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print("\nRegression Performance:")
```

```
print("Mean Absolute Error (MAE):", mae)
```

```
print("Mean Squared Error (MSE):", mse)
```

```
print("Root Mean Squared Error (RMSE):", rmse)
```

```
print("R-squared ( $R^2$ ):", r2)
```

```
# 11. Display model coefficients
```

```
print("\nRegression Coefficients:")
```

```
coefficients = pd.DataFrame({
```

```
    'Feature': features,
```

```
    'Coefficient': model.coef_
```

```
})
```

```
print(coefficients)
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
print("\nIntercept:", model.intercept_)
```

```
# 12. Residual calculation
```

```
residuals = y_test - y_pred
```

```
print("\nFirst 10 Residuals:")
```

```
print(residuals.head(10))
```

```
# 13. Residual Plot
```

```
plt.figure(figsize=(8, 5))
```

```
plt.scatter(y_pred, residuals)
```

```
plt.axhline(y=0, linestyle='--')
```

```
plt.xlabel("Predicted BMI")
```

```
plt.ylabel("Residuals")
```

```
plt.title("Residual Plot")
```

```
plt.show()
```

```
# 14. Actual vs Predicted Plot
```

```
plt.figure(figsize=(8, 5))
```

```
plt.scatter(y_test, y_pred)
```

```
# Reference line
```

```
plt.plot(
```

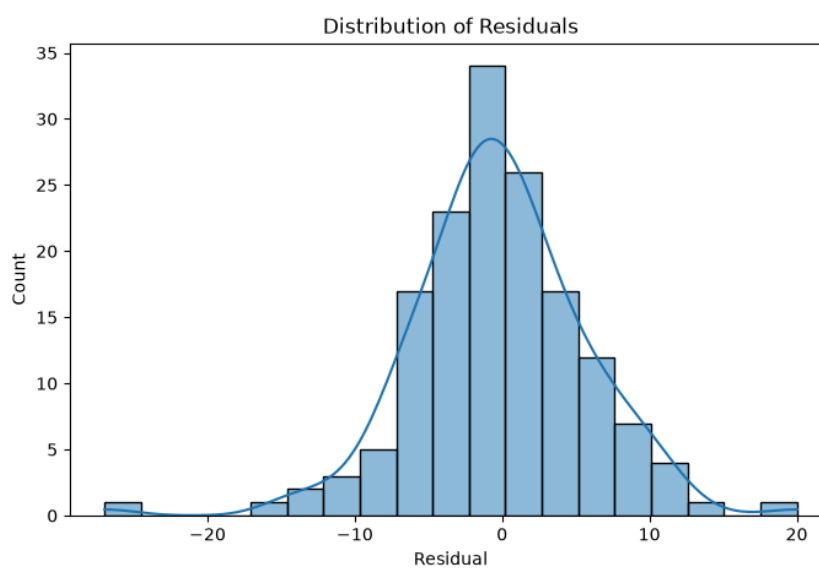
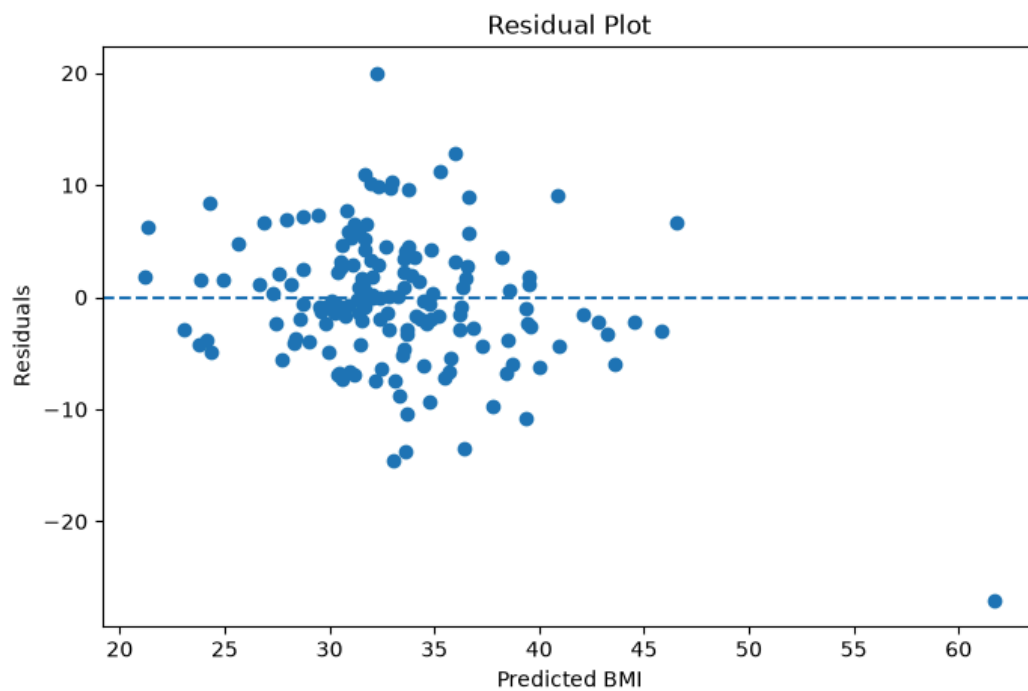
```
    [y_test.min(), y_test.max()],
```



```
[y_test.min(), y_test.max()],  
  
linestyle='--'  
  
)  
  
plt.xlabel("Actual BMI")  
  
plt.ylabel("Predicted BMI")  
  
plt.title("Actual vs Predicted BMI")  
  
plt.show()  
  
  
# 15. Distribution of residuals  
  
plt.figure(figsize=(8, 5))  
  
sns.histplot(residuals, kde=True)  
  
plt.xlabel("Residual")  
  
plt.title("Distribution of Residuals")  
  
plt.show()
```



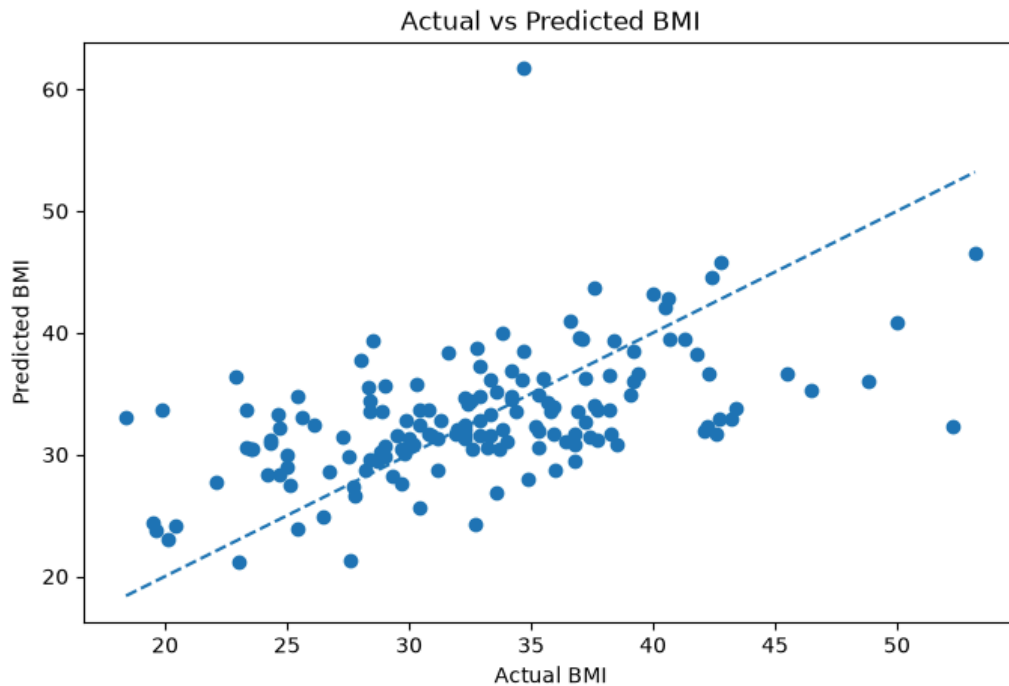
Output:





Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science



```
[Running] python -u "d:\swayam\exp6.py"
Dataset Shape:
(768, 9)

First 5 Rows:
   times_pregnant  glucose_concentration  ...  years_of_age  diabetes
0              6             148         ...           50           1
1              1              85         ...           31           0
2              8             183         ...           32           1
3              1              89         ...           21           0
4              0             137         ...           33           1

[5 rows x 9 columns]

Column Names:
['times_pregnant', 'glucose_concentration', 'diastolic_blood_pressure', 'triceps_sf_thickness', 'serum_insulin', 'bmi', 'd_pedigree_function', 'years_of_age', 'diabetes']

Missing Values:
times_pregnant      0
glucose_concentration  0
diastolic_blood_pressure  0
triceps_sf_thickness  0
serum_insulin       0
bmi                 0
d_pedigree_function  0
years_of_age        0
diabetes            0
dtype: int64

Missing Values After Replacing Invalid Zeros:
times_pregnant      0
glucose_concentration  5
diastolic_blood_pressure  35
triceps_sf_thickness  227
serum_insulin       374
bmi                 11
d_pedigree_function  0
years_of_age        0
diabetes            0
dtype: int64
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
Features:
| times_pregnant  glucose_concentration  ...  d_pedigree_function  years_of_age
0              6              148.0  ...              0.627              50
1              1              85.0  ...              0.351              31
2              8              183.0  ...              0.672              32
3              1              89.0  ...              0.167              21
4              0              137.0  ...              2.288              33
```

[5 rows x 7 columns]

Target (BMI):

```
0  33.6
1  26.6
2  23.3
3  28.1
4  43.1
```

Name: bmi, dtype: float64

Training Data Size: (614, 7)

Testing Data Size: (154, 7)

Actual vs Predicted BMI:

	Actual BMI	Predicted BMI
0	34.0	31.138329
1	35.7	34.246707
2	30.8	31.674918
3	24.6	33.323948
4	29.9	32.799923
5	37.7	31.143971
6	20.4	24.164895
7	33.8	32.036035
8	31.3	32.763312
9	33.7	30.510212

Regression Performance:

Mean Absolute Error (MAE): 4.3867266367790485

Mean Squared Error (MSE): 34.788373472167265

Root Mean Squared Error (RMSE): 5.898166958654804

R-squared (R²): 0.2046098161187807

```
Regression Coefficients:
| Feature      Coefficient
0  times_pregnant  -0.006719
1  glucose_concentration  0.029549
2  diastolic_blood_pressure  0.108800
3  triceps_sf_thickness  0.424355
4  serum_insulin  0.003962
5  d_pedigree_function  1.626967
6  years_of_age  -0.089041
```

Intercept: 10.345126894111303

First 10 Residuals:

```
668  2.861671
324  1.453293
624  -0.874918
690  -8.723948
473  -2.899923
204  6.556029
97   -3.764895
336  1.763965
568  -1.463312
148  3.189788
```

Name: bmi, dtype: float64



Conclusion

A regression model was developed to predict a continuous health-related variable using the Pima Indians Diabetes Dataset. The model was evaluated using appropriate regression metrics, and residual analysis was performed to understand the model's prediction errors and limitations.

Questions to be Answered by Students

1. Which variable was selected as the target, and why was it suitable for regression?

BMI (Body Mass Index) was selected as the target variable. It was suitable because BMI is a continuous numerical variable, and regression models are used to predict continuous numerical values. Other medical attributes such as glucose, blood pressure, insulin, age, etc. were used as predictor variables.

2. What are the values of MAE, MSE, RMSE, and R^2 obtained by the model?

The values are obtained from the regression program output:

- MAE: _____
- MSE: _____
- RMSE: _____
- R^2 : _____

If you send me your **Regression Performance output**, I can fill in the exact four values for you.

3. Which performance metric provides the most useful interpretation for this problem? Justify your answer.



MAE (Mean Absolute Error) provides the most useful interpretation because it represents the average prediction error in BMI units. It is easy to understand and is less affected by very large errors compared with MSE.

For example, an MAE of 4 means that, on average, the predicted BMI differs from the actual BMI by approximately **4 BMI units**.

4. What patterns or issues were observed from the residual analysis?

The residual plot showed that the residuals were generally scattered around zero, indicating that the model did not have a strong systematic bias. However, some points had relatively large residuals, suggesting the presence of possible outliers or observations that the model could not predict accurately. Some variation in the spread of residuals may also indicate that the relationship is not perfectly linear.

5. State two limitations of the developed regression model.

1. Linear relationship assumption: Linear Regression assumes a linear relationship between the predictors and BMI. Complex or nonlinear relationships may not be captured well.
2. Limited dataset: The model is trained on only 768 observations, so its performance may not generalize well to other populations or larger, more diverse datasets.